

```
In [1]: from pyspark.sql import SparkSession
```

```
In [2]: spark = SparkSession.builder \
        .appName("PhoneValidation") \
        .master("local[*]") \
        .getOrCreate()
```

```
In [3]: from pyspark.sql import functions as F
        from pyspark.sql.window import Window
```

Read PHONE and MEMBER tables into DataFrames.

```
In [4]: phone_data = [
        (1, 101, "9123456789", "2024-01-01 09:00:00"),
        (2, 101, "9345678901", "2024-01-15 10:00:00"),
        (3, 102, "1234509876", "2024-01-20 11:00:00"),
        (4, 103, "9555512345", "2024-01-25 07:00:00")
        # (9, 101, "9123456789", "2024-02-01 09:00:00"),
        # (10, 101, "9345688901", "2024-02-02 10:00:00"),
        # (11, 102, "1234509876", "2024-02-03 11:00:00"),
        # (12, 102, "9988776655", "2024-02-04 08:00:00"),
        # (13, 103, "9555512345", "2024-02-05 07:00:00"),
        # (14, 104, "9871234567", "2024-02-06 06:00:00"),
        # (15, 104, "98AB234567", "2024-02-06 07:00:00")
        ]

        member_data = [
        (101, 1001),
        (102, 1002),
        (103, 1003),
        (104, 1004)
        ]

        phone_df = spark.createDataFrame(phone_data, ["phone_id", "mbr_id", "phone_number", "last_updated_dt"])
        member_df = spark.createDataFrame(member_data, ["mbr_id", "person_id"])
```

Join on mbr_id to get person_id.

```
In [5]: df = phone_df.join(member_df, on="mbr_id", how="inner")
        df.show()
```

```
+-----+-----+-----+-----+-----+
|mbr_id|phone_id|phone_number|last_updated_dt|person_id|
+-----+-----+-----+-----+-----+
|  101|      1| 9123456789|2024-01-01 09:00:00|    1001|
|  101|      2| 9345678901|2024-01-15 10:00:00|    1001|
|  102|      3| 1234509876|2024-01-20 11:00:00|    1002|
|  103|      4| 9555512345|2024-01-25 07:00:00|    1003|
+-----+-----+-----+-----+-----+
```

Validate phone numbers using rules:-

Must be 10 digits.- Must be numeric.- Must not start with 0 or 1

Mark records as VALID or INVALID

```
In [6]: df = df.withColumn(
        "is_valid",
        F.when(
            (F.col("phone_number").rlike("^[2-9][0-9]{9}$")),
            "VALID"
        ).otherwise("INVALID")
        )
        df.show()
```

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid
101	1	9123456789	2024-01-01 09:00:00	1001	VALID
101	2	9345678901	2024-01-15 10:00:00	1001	VALID
102	3	1234509876	2024-01-20 11:00:00	1002	INVALID
103	4	9555512345	2024-01-25 07:00:00	1003	VALID

Filter only valid records.

```
In [7]: valid_df = df.filter(F.col("is_valid") == "VALID")
        valid_df.show()
```

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid
101	1	9123456789	2024-01-01 09:00:00	1001	VALID
101	2	9345678901	2024-01-15 10:00:00	1001	VALID
103	4	9555512345	2024-01-25 07:00:00	1003	VALID

Use Window function:-

Partition by person_id- Order by last_updated_dt DESC

```
In [8]: window_spec = Window.partitionBy("person_id").orderBy(F.col("last_updated_dt").desc())

        valid_df = valid_df.withColumn(
            "rank",
            F.row_number().over(window_spec)
        )
```

Assign Primary (Y) to latest record, others Secondary (N).

```
In [9]: processed_df = valid_df.withColumn(
        "primary_flag",
        F.when(F.col("rank") == 1, "Y").otherwise("N")
    ).drop("rank")

        processed_df.show()
```

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid	primary_flag
101	2	9345678901	2024-01-15 10:00:00	1001	VALID	Y
101	1	9123456789	2024-01-01 09:00:00	1001	VALID	N
103	4	9555512345	2024-01-25 07:00:00	1003	VALID	Y

Create temp view for processed data

```
In [10]: processed_df.createOrReplaceTempView("processed_phone_view")
         spark.sql("SELECT * FROM processed_phone_view").show()
```

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid	primary_flag
101	2	9345678901	2024-01-15 10:00:00	1001	VALID	Y
101	1	9123456789	2024-01-01 09:00:00	1001	VALID	N
103	4	9555512345	2024-01-25 07:00:00	1003	VALID	Y

Merge with final table and deduplicate using latest timestamp.

```
In [11]: try:
existing_df = spark.read.option("header", True).option("inferSchema", True).csv("phone_final_table.csv")
except:
existing_df = spark.createDataFrame([], processed_df.schema)

print("Existing Record : ")
existing_df.show()

union_df = existing_df.unionByName(processed_df, allowMissingColumns=True)

window_spec = Window.partitionBy("person_id")\
                    .orderBy(F.col("last_updated_dt").desc())

dedup_window = Window.partitionBy("person_id", "phone_number")\
                    .orderBy(F.col("last_updated_dt").desc())

deduped_df = (
    union_df
    .withColumn("rank", F.row_number().over(dedup_window))
    .filter(F.col("rank") == 1)
    .drop("rank")
    .withColumn("primary_flag", F.when(F.row_number().over(window_spec) == 1, "Y").otherwise("N"))
)
```

Existing Record :

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid	primary_flag
101	2	9345678901	2024-01-15 10:00:00	1001	VALID	Y
101	1	9123456789	2024-01-01 09:00:00	1001	VALID	N
103	4	9555512345	2024-01-25 07:00:00	1003	VALID	Y

Write results to final table.

```
In [12]: deduped_df.write.mode("overwrite").option("header", True).csv("phone_final_table.csv")
df = spark.read.option("header", True).option("inferSchema", True).csv("phone_final_table.csv")
df.show()
```

mbr_id	phone_id	phone_number	last_updated_dt	person_id	is_valid	primary_flag
101	2	9345678901	2024-01-15 10:00:00	1001	VALID	Y
101	1	9123456789	2024-01-01 09:00:00	1001	VALID	N
103	4	9555512345	2024-01-25 07:00:00	1003	VALID	Y